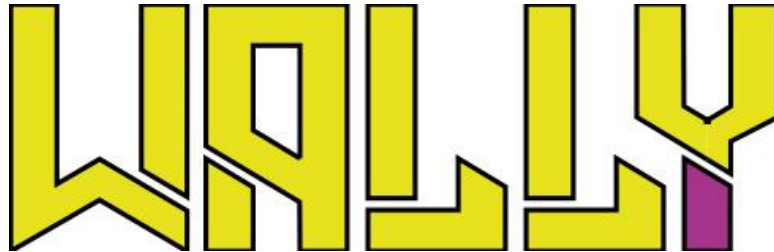




Professionele Bachelor Toegepaste Informatica



Real-time update-strategieën voor schaalbare robotmonitoring in webdashboards

Mathias Slegers

Promotoren:

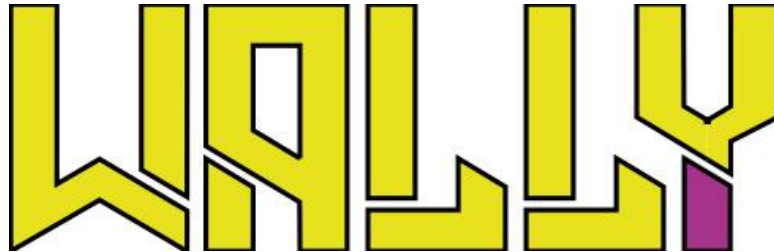
Maarten Gielkens
Stany Smets

Wally Automation
Hogeschool PXL Hasselt





Professionele Bachelor Toegepaste Informatica



Real-time update-strategieën voor schaalbare robotmonitoring in webdashboards

Mathias Slegers

Promotoren:

Maarten Gielkens

Wally Automation

Stany Smets

Hogeschool PXL Hasselt



Dankwoord

Graag wil ik enkele personen bedanken die mij tijdens mijn stage en het schrijven van dit eindwerk hebben ondersteund.

In de eerste plaats wil ik Wally Automation bedanken voor de kans om mijn stage binnen het bedrijf uit te voeren. De stage bood mij de mogelijkheid om mee te werken aan een technisch uitdagend project binnen een innovatieve en praktijkgerichte omgeving. Daarbij kreeg ik de ruimte om zelfstandig te werken, technische keuzes te onderzoeken en mijn kennis verder te ontwikkelen.

Daarnaast wil ik mijn bedrijfspromotor, Maarten Gielkens, bedanken voor de begeleiding, feedback en ondersteuning tijdens de stage. Zijn technische inzichten en bereidheid om mee na te denken over oplossingen hebben een belangrijke bijdrage geleverd aan de uitwerking van mijn stageopdracht.

Ook wil ik het volledige team van Wally Automation bedanken voor de aangename samenwerking, de open communicatie en de hulp bij technische vragen. De korte communicatielijnen en de betrokkenheid binnen het team hebben ervoor gezorgd dat ik snel kon bijleren en actief kon bijdragen aan het project.

Verder bedank ik mijn hogeschoolpromotor, Stany Smets, voor de opvolging, feedback en begeleiding vanuit Hogeschool PXL. Zijn opmerkingen hielpen om de scope van het onderzoek duidelijker af te bakenen en het eindwerk verder te structureren.

Abstract

Stageopdracht

De stage vindt plaats bij Wally Automation, een start-up die een autonome metselrobot ontwikkelt voor de bouwsector. Deze robot genereert continu operationele data, zoals productiviteit, foutmeldingen en statusinformatie, die essentieel zijn voor de monitoring en analyse van het bouwproces. De stage richt zich op de ontwikkeling van een schaalbare webapplicatie om robotdata te visualiseren.

De robots slaan operationele data zelfstandig op in een centrale database. Daarnaast wordt een aparte datastroom opgezet die real-time gegevens naar de webapplicatie stuurt. Deze datastroom wordt uitsluitend gebruikt voor real-time monitoring en heeft geen invloed op de werking of besturing van de robot.

De webapplicatie visualiseert zowel real-time als historische data en biedt inzicht in de prestaties en eventuele fouten van de robot. Op zowel robot- als projectniveau worden relevante gegevens geanalyseerd en weergegeven om inefficiënties in het bouwproces snel te detecteren.

Onderzoeksopdracht

Het onderzoek richt zich op het bepalen van de meest geschikte real-time-updatestrategie voor een schaalbaar webdashboard om robots te monitoren. Verschillende technieken, zoals polling en pushgebaseerde communicatie, worden vergeleken op basis van latentie, schaalbaarheid en netwerkbelasting. Hierbij wordt rekening gehouden met de soft real-time vereisten van het systeem. Het doel is een optimale balans te vinden tussen performantie, eenvoud en bruikbaarheid.

Inhoudsopgave

Dankwoord	ii
Abstract	iii
Inhoudsopgave	iv
Lijst van gebruikte figuren.....	vi
Lijst van gebruikte tabellen	vii
Lijst van gebruikte afkortingen	viii
Inleiding.....	1
I. Stageverslag	2
1 Bedrijfsvoorstelling	2
2 Stageopdracht	3
3 Stageverloop	4
3.1 Analyse van de bestaande applicatie en dataflow	4
3.2 Herwerking van de projectstructuur en states.....	4
3.3 Ontwikkeling van robot- en projectvisualisaties	5
II. Onderzoekstopic	6
4 Probleemstelling	6
5 Onderzoeksvraag & Hypothese.....	7
5.1 Onderzoeksvraag:	7
5.2 Hypothese	7
III. Methodologie.....	8
IV. Literatuurstudie.....	9
6 Analyse van Real-time Vereisten binnen de Context van Autonome Metselrobots	9
6.1 Functioneel responstijden voor webgebaseerde dashboards	10
7 Technologische Verkenning en Impactanalyse van Update-strategieën.....	11
7.1 Polling	12
7.1.1 Short polling	12
7.1.2 Long polling	12
7.1.3 Vergelijkende analyse en toepasbaarheid	12
7.2 Server Push (SSE)	14
7.2.1 SSE binnen HTTP/1.1	14
7.2.2 SSE binnen HTTP/2	14
7.2.3 Positionering ten opzichte van andere strategieën	14

7.3	Bidirectioneel.....	15
7.3.1	WebSockets	15
7.3.2	MQTT (Message Queuing Telemetry Transport)	17
7.3.3	Werking en architectuur	17
7.3.4	Quality of Service (QoS).....	17
7.3.5	Efficiëntie en performantie	18
7.3.6	Betrouwbaarheid en robuustheid	18
7.3.7	Positionering binnen de architectuur van Wally Automation	19
7.3.8	Evaluatie ten opzichte van andere strategieën.....	19
7.4	gRPC (Google Remote Procedure Call)	20
7.4.1	Werking en architectuur	20
7.4.2	Communicatiepatronen	20
7.4.3	Prestaties en efficiëntie.....	21
7.4.4	Positionering binnen de architectuur van Wally Automation	21
8	Synthese en aanzet tot het primair onderzoek	22
V.	Primair onderzoek	23
	Conclusie	23
	Reflectie.....	24
	Bronnenlijst	25
	Bijlagen	26

Lijst van gebruikte figuren

Figuur 1 MQTT publish-subscribe architectuur	17
--	----

Lijst van gebruikte tabellen

Tabel 1 Vergelijking van short polling en long polling	13
--	----

Lijst van gebruikte afkortingen

▪ API	Application Programming Interface
▪ CPU	Central Processing Unit
▪ gRPC	Google Remote Procedure Call
▪ HTML	HyperText Markup Language
▪ HTTP	Hypertext Transfer Protocol
▪ IDL	Interface Definition Language
▪ IETF	Internet Engineering Task Force
▪ IoT	Internet of Things
▪ IT	Information Technology
▪ JSON	JavaScript Object Notation
▪ KPI	Key Performance Indicator
▪ LWT	Last Will and Testament
▪ MQTT	Message Queuing Telemetry Transport
▪ OASIS	Organization for the Advancement of Structured Information Standards
▪ PoC	Proof of Concept
▪ Protobuf	Protocol Buffers
▪ QoS	Quality of Service
▪ QUIC	Quick UDP Internet Connections
▪ REST	Representational State Transfer
▪ RFC	Request for Comments
▪ RPC	Remote Procedure Call
▪ SOAP	Simple Object Access Protocol
▪ SSE	Server-Sent Events
▪ TCP	Transmission Control Protocol
▪ TLS	Transport Layer Security
▪ USP	Unique Selling Proposition
▪ W3C	World Wide Web Consortium
▪ WebRTC	Web Real-Time Communication
▪ WHATWG	Web Hypertext Application Technology Working Group

Inleiding

Dit eindwerk kadert binnen de bachelorproef van de opleiding Toegepaste Informatica. Het beschrijft de stage die werd uitgevoerd bij Wally Automation, een Belgische start-up die autonome metselrobots ontwikkelt voor de bouwsector. Binnen deze stage wordt gewerkt aan een schaalbare webapplicatie voor het visualiseren van operationele robotdata.

De autonome metselrobot genereert tijdens zijn werking verschillende soorten data, zoals statusinformatie, productiviteit, foutmeldingen en projectgebonden gegevens. Om deze informatie bruikbaar te maken voor opvolging en analyse, is een centrale visualisatielaag nodig. De stageopdracht richt zich daarom op de ontwikkeling van een webapplicatie die zowel actuele als historische robotdata overzichtelijk weergeeft.

Naast de praktische stageopdracht bevat dit eindwerk ook een onderzoeksrapport. Het onderzoek gaat na welke real-time updatestrategie het meest geschikt is voor een schaalbaar webdashboard voor robotmonitoring. Daarbij worden verschillende strategieën vergeleken op basis van onder meer latentie, schaalbaarheid, netwerkbelasting en onderhoudbaarheid.

Het eindwerk bestaat uit twee grote delen. Het eerste deel bevat het stageverslag. Daarin worden het stagebedrijf, de stageopdracht, het verloop van de stage, de uitgevoerde werkzaamheden en het opgeleverde resultaat besproken. Het tweede deel bevat het onderzoeksrapport. Daarin komen de probleemstelling, onderzoeksvragen, methodologie, literatuurstudie, resultaten, discussie en conclusie aan bod.

I. Stageverslag

1 Bedrijfsvoorstelling

Wally Automation is een Belgische start-up actief in de bouwrobotica. Het bedrijf werd opgericht in oktober 2024 en is gevestigd in Genk, met kantoorruimte in IncubaThor op Thor Park. Wally Automation bevindt zich in een vroege groeifase en focust op de automatisering van muurconstructies via autonome robotica.

Een belangrijk deel van de technologische ontwikkeling en validatie vindt plaats in samenwerking met ConstrucThor van KU Leuven, binnen het kader van het Open Thor Living Lab. Deze samenwerking positioneert het bedrijf binnen een onderzoeksgerichte omgeving waar innovatie, experimentele validatie en praktijkgerichte toepassingen centraal staan.

De kernactiviteit van Wally Automation situeert zich op het snijvlak van robotica, softwareontwikkeling en de bouwsector. De belangrijkste unique selling proposition (USP) op IT-vlak is de ontwikkeling van een geïntegreerd softwaresysteem dat digitale bouwplannen omzet in nauwkeurige en uitvoerbare instructies voor een autonome metselrobot. Hierdoor wordt een brug geslagen tussen digitale ontwerpdata en de fysieke uitvoering op de werf.

Wally Automation bestaat uit een kernteam van vier vaste medewerkers, aangevuld met een aantal stagiairs die actief bijdragen aan zowel softwareontwikkeling als onderzoek. Gezien de beperkte schaal van de organisatie is er geen afzonderlijke IT-afdeling of complexe hiërarchische structuur; alle teamleden zijn rechtstreeks betrokken bij de technologische ontwikkeling.

De organisatie wordt gekenmerkt door een vlakke structuur en korte communicatielijnen. Besluitvorming verloopt efficiënt en technologische innovaties kunnen snel worden getest en geïmplementeerd. Deze wendbaarheid is kenmerkend voor een start-up in een vroege ontwikkelingsfase.

2 Stageopdracht

Wally Automation ontwikkelt een autonome metselrobot voor de bouwsector. Deze robot genereert tijdens zijn werking verschillende soorten operationele data, zoals productiviteit, statusinformatie, foutmeldingen, stilstanden en informatie over het verbruik en de kwaliteit van lijm. Deze gegevens zijn belangrijk om de werking van de robot op te volgen, problemen sneller te detecteren en het bouwproces beter te analyseren.

Binnen de stage wordt gewerkt aan een schaalbare webapplicatie voor het visualiseren van robotdata. De applicatie moet zowel actuele als historische gegevens overzichtelijk weergeven. Daardoor krijgen gebruikers inzicht in de prestaties, foutmeldingen en algemene status van de robot. De visualisaties worden uitgewerkt op robotniveau en projectniveau, zodat zowel individuele robotprestaties als bredere projectinformatie opgevolgd kunnen worden.

De opdracht wordt gerealiseerd voor Wally Automation en ondersteunt voornamelijk de interne ontwikkelaars, operatoren en projectverantwoordelijken. Zij hebben nood aan een centrale omgeving waarin robotstatus, productiviteit en foutinformatie snel raadpleegbaar zijn. Zonder een dergelijke visualisatielaag is het moeilijker om operationele problemen tijdig te detecteren en om prestaties over langere periodes te analyseren.

De robotdata worden opgeslagen in een centrale database. Daarnaast wordt een aparte datastroom opgezet die real-time gegevens naar de webapplicatie stuurt. Deze datastroom wordt uitsluitend gebruikt voor monitoring en visualisatie. Ze heeft geen invloed op de werking of besturing van de robot. Op die manier blijft de monitoringlaag gescheiden van de operationele robotsturing.

De technische omgeving van de stage bestaat uit verschillende onderdelen die samen de basis vormen voor het dataplatform. De robotomgeving maakt gebruik van Linux, ROS2 en Python. Om data op te slaan en te verwerken, wordt een centrale database ingezet, waarbij MongoDB als belangrijke technologie wordt gebruikt. De webapplicatie vormt de visualisatielaag bovenop deze data-infrastructuur.

Naast de praktische stageopdracht sluit ook de onderzoeksoopdracht aan bij dit thema. Het onderzoek focust op de manier waarop real-time gegevens efficiënt naar het dashboard kunnen worden gestuurd. De resultaten van dit onderzoek ondersteunen de technische keuzes binnen de webapplicatie, met bijzondere aandacht voor performantie, eenvoud en onderhoudbaarheid.

3 Stageverloop

Dit hoofdstuk beschrijft de uitwerking van de stageopdracht. De nadruk ligt op de aanpak en implementatie van het monitoringdashboard, met aandacht voor de analyse van de bestaande applicatie, de herwerking van de datastructuur, de ontwikkeling van visualisaties en de eerste integratie van real-time communicatie.

3.1 Analyse van de bestaande applicatie en dataflow

In de eerste fase van de stage wordt de bestaande applicatie van Wally Automation geanalyseerd. Daarbij ligt de focus op de manier waarop robotdata en projectdata worden opgeslagen, verwerkt en weergegeven. De bestaande codebase wordt onderzocht om te bepalen welke onderdelen behouden kunnen blijven en welke onderdelen aangepast moeten worden om het monitoringdashboard verder uit te bouwen.

Tijdens deze analyse wordt ook nagegaan welke gegevens relevant zijn voor de visualisatie van robot- en projectinformatie. De stageopdracht vertrekt vanuit de nood om operationele robotdata, zoals geplaatste stenen, productiviteit, lijmgegevens, stilstanden en foutcodes, centraal beschikbaar te maken via interactieve dashboards. Daarom is het belangrijk om eerst inzicht te krijgen in de bestaande dataflow en in de structuur van de huidige applicatie.

Naast de analyse van de codebase worden ook de eerste voorbereidende aanpassingen uitgevoerd. Zo wordt gestart met de basisimplementatie van visualisatiecomponenten, waaronder een 2D- en 3D-planviewer. Deze componenten vormen de basis voor de latere robot- en projectvisualisaties.

3.2 Herwerking van de projectstructuur en states

Tijdens de verdere ontwikkeling wordt duidelijk dat de bestaande structuur voor projectstates beperkingen heeft. Het actieve project van een robot kan niet eenvoudig genoeg worden uitgelezen. Daarnaast maakt de bestaande structuur het moeilijker om de historie van een project duidelijk weer te geven.

Om dit probleem op te lossen, wordt het states-systeem herwerkt. Daarbij wordt de levenscyclus van een project duidelijker gestructureerd, zodat de status van een project eenvoudiger opgevolgd kan worden. Deze aanpassing maakt het mogelijk om projectinformatie consistent te gebruiken binnen de applicatie.

Naast de herwerking van de states wordt ook een event-gebaseerde aanpak toegevoegd. Projectgebeurtenissen worden opgeslagen als afzonderlijke events in de database. Daardoor kan de historie van een project later eenvoudiger worden weergegeven en geanalyseerd. Deze aanpak biedt ook mogelijkheden voor tijdsanalyses, bijvoorbeeld om te onderzoeken wanneer bepaalde stappen in het bouwproces plaatsvinden of waar vertragingen ontstaan.

3.3 Ontwikkeling van robot- en projectvisualisaties

Na de analyse en de herwerking van de projectstructuur wordt verder gewerkt aan de robotdetailpagina. Deze pagina heeft als doel om de toestand van een robot en het gekoppelde project overzichtelijk weer te geven. De visualisaties moeten gebruikers helpen om sneller inzicht te krijgen in de voortgang, status en mogelijke problemen.

Een belangrijk onderdeel hiervan is de 3D-weergave. Deze wordt aangepast zodat geplaatste en nog niet geplaatste stenen visueel van elkaar onderscheiden kunnen worden. Daardoor wordt de voortgang van een muurconstructie duidelijker zichtbaar binnen het dashboard. In dezelfde fase worden ook de projectentiteiten aangepast, waarbij muren in een aparte collectie worden opgeslagen. Deze structuur maakt het eenvoudiger om muren, stenen en projectgegevens afzonderlijk te beheren en te visualiseren.

II. Onderzoekstopic

4 Probleemstelling

Wally Automation ontwikkelt autonome metselrobots die tijdens het bouwproces grote hoeveelheden operationele data genereren. Deze data worden vanuit de robot opgeslagen in een MongoDB-database en omvatten onder meer informatie over robotactiviteit, voortgang en projectuitvoering. Daarnaast wordt ook projectgebonden data verzameld, zoals doorlooptijd, geolocatie en andere variabelen die relevant zijn voor het opvolgen van vertragingen, inefficiënties en operationele knelpunten. Deze gegevens vormen een belangrijke basis voor monitoring en analyse binnen het bedrijf.

Om deze informatie bruikbaar te maken voor interne opvolging, is een webgebaseerd monitoringdashboard nodig. Zo'n dashboard moet project- en robotdata op een duidelijke en tijdige manier visualiseren, zodat gebruikers de status van robots en projecten kunnen opvolgen en sneller kunnen reageren op afwijkingen of problemen. Hierbij stelt zich echter een technisch vraagstuk: niet alle data heeft dezelfde updatefrequentie of dezelfde nood aan onmiddellijke weergave. Sommige gegevens moeten zo actueel mogelijk zichtbaar zijn, terwijl andere informatie minder tijdsgevoelig is.

De uitdaging is daarom te bepalen welke real-time update-strategie het meest geschikt is voor dit type monitoringplatform. Verschillende strategieën, zoals polling, push-gebaseerde communicatie of een combinatie van beide, kunnen hierin een rol spelen. Elke strategie heeft mogelijke gevolgen voor *end-to-end* latentie, serverbelasting, netwerkverkeer, schaalbaarheid en de manier waarop data beschikbaar blijft of opnieuw gesynchroniseerd wordt bij tijdelijk netwerkverlies. Daarnaast heeft de gekozen update-aanpak ook invloed op de bruikbaarheid en responsiviteit van de visualisaties in het dashboard.

Vanuit deze context ontstaat de nood om verschillende real-time update-strategieën systematisch te onderzoeken en te vergelijken binnen de specifieke context van een schaalbaar webgebaseerd monitoringdashboard voor autonome metselrobots. Dit sluit aan bij de stageopdracht, waarin operationele robotdata gecentraliseerd, verwerkt en gevisualiseerd moet worden binnen een intern platform voor monitoring en controle.

5 Onderzoeksvraag en hypothese

5.1 Onderzoeksvraag

Welke real-time update-strategie is het meest geschikt voor een schaalbaar webgebaseerd monitoringdashboard voor autonome metselrobots, rekening houdend met latentie, serverbelasting, netwerkverkeer, schaalbaarheid en de bruikbaarheid van de visualisaties?

Deelvragen:

- Wat betekent “real-time” binnen de context van robotmonitoring en welke responstijden zijn functioneel vereist voor een monitoringdashboard?
- Welke real-time update-strategieën zijn geschikt voor webgebaseerde monitoringdashboards voor robotdata?
- Wat is de impact van deze update-strategieën op:
 - End-to-end latentie
 - Serverbelasting (CPU/geheugen)
 - Netwerkverkeer
 - Dataretentie bij netwerkverlies
- Hoe gedragen de verschillende update-strategieën zich wanneer het aantal gelijktijdige robotdatastromen toeneemt?
- Welke invloed hebben de verschillende update-strategieën op de responsiviteit en bruikbaarheid van de visualisaties in het dashboard?
- Welke real-time update-strategie biedt de beste balans tussen technische eenvoud, schaalbaarheid en toekomstbestendigheid voor het monitoringplatform van Wally Automation? (Conclusie)

5.2 Hypothese

Er wordt verwacht dat een hybride real-time update-strategie, waarbij event-gedreven data via WebSockets wordt verzonden en minder tijdkritische status- en KPI-informatie via periodieke polling wordt opgehaald, de beste balans zal bieden tussen lage latentie, serverperformantie, schaalbaarheid en bruikbaarheid voor een monitoringdashboard van autonome metselrobots.

III. Methodologie

Dit hoofdstuk beschrijft de stappen die worden ondernomen om de onderzoeksvraag te beantwoorden. Het onderzoek bestaat uit twee delen: een theoretische verkenning via een literatuurstudie en een praktische validatie door middel van een Proof of Concept (PoC). Het doel is niet om een volledig afgewerkt product te ontwikkelen, maar om een gecontroleerde testomgeving op te zetten waarin verschillende update-strategieën objectief met elkaar kunnen worden vergeleken.

Literatuurstudie

De literatuurstudie vormt het startpunt van het onderzoek. Hierin worden de vereisten voor soft real-time monitoringdashboards in kaart gebracht en worden de technische eigenschappen van verschillende update-mechanismen, zoals polling, Server-Sent Events (SSE) en WebSockets, geanalyseerd. Op basis van deze theoretische inzichten wordt een centrale hypothese geformuleerd, namelijk dat een hybride strategie de beste balans biedt tussen latentie, betrouwbaarheid en efficiëntie.

Primair onderzoek

Om real-time update-strategieën te vergelijken, wordt een gesimuleerde eventbron ontwikkeld die het gedrag van een robot nabootst door op gecontroleerde wijze events te genereren. Deze events worden via verschillende communicatiekanalen, namelijk MQTT, WebSockets, gRPC, Server-Sent Events (SSE) en *polling*, afzonderlijk verwerkt en doorgestuurd naar een webapplicatie.

Per strategie wordt een identieke dataset gebruikt om een eerlijke vergelijking te garanderen. Tijdens de experimenten worden meerdere scenario's getest met variërende eventfrequenties en payloadgroottes. Voor elk event worden timestamps geregistreerd bij creatie en ontvangst, zodat de *end-to-end* latentie en spreiding kunnen worden berekend. Daarnaast worden ook het netwerkverbruik, de betrouwbaarheid (zoals verloren of dubbele events) en de volgorde van aflevering geanalyseerd. De resultaten van deze metingen vormen de basis voor een onderbouwde vergelijking van de verschillende update-strategieën.

IV. Literatuurstudie

6 Analyse van real-time vereisten binnen de context van autonome metselrobots

In robotica verwijst de term “real-time” niet noodzakelijk naar reacties binnen milliseconden, maar of informatie beschikbaar is op het moment dat ze functioneel vereist is. De correctheid van een real-time systeem wordt daarom bepaald door zowel de logische juistheid als de tijdige oplevering van resultaten [1].

Binnen robotica en monitoring kan een onderscheid worden gemaakt tussen twee vormen van real-time systemen [1], [4].

- **Hard real-time:**
Dit type systemen wordt toegepast in missiekritieke onderdelen, zoals de interne besturing van een robot. In deze context leidt een te late reactie tot het volledig falen van de functionaliteit, wat mogelijk schade of gevaarlijke situaties veroorzaakt. Tijdigheid is hier een absolute vereiste [1].
- **Soft real-time:**
Webgebaseerde monitoringtoepassingen vallen doorgaans in deze categorie. De waarde van informatie neemt af naarmate deze later beschikbaar komt, maar kleine vertragingen leiden niet onmiddellijk tot systeemfalen [1], [4].

Voor een monitoringdashboard voor autonome metselrobots is soft real-time het meest relevante paradigma. Het dashboard maakt geen deel uit van de primaire controlelus van de robot, maar fungeert als een supervisie- en diagnose-interface voor projectverantwoordelijken. De stabiliteit en veiligheid van de robot worden lokaal gegarandeerd, terwijl het dashboard voornamelijk dient voor observatie, analyse en besluitvorming.

Om een soft real-time ervaring in een webomgeving te realiseren, wordt gebruikgemaakt van push-gebaseerde communicatiemechanismen met lage latentie, zoals WebSockets (voor bidirectionele communicatie) of Server-Sent Events (voor server-naar-client updates). Deze technieken vermijden de inefficiënties van periodieke polling, waarbij de client op vaste intervallen data opvraagt en daardoor extra latentie introduceert [2], [4].

6.1 Functionele responstijden voor webgebaseerde dashboards

Hoewel het systeem als webapplicatie wordt geïmplementeerd, worden de vereiste responstijden voornamelijk bepaald door menselijke perceptie en gebruikerservaring, en niet door de onderliggende technologie. In dit kader kunnen drie belangrijke drempels worden onderscheiden [3]:

- **0,1 seconde - onmiddellijke respons:**
Reacties binnen deze tijdsgrens worden door de gebruiker als direct ervaren. Dit is relevant voor interacties zoals het selecteren van een robot, het aanpassen van filters of het navigeren binnen het dashboard. Extra visuele feedback is in deze situatie niet noodzakelijk.
- **1 seconde - behoud van cognitieve flow:**
Binnen deze grens blijft de gebruiker gefocust op de taak, ondanks een merkbare vertraging. Voor operaties die langer duren dan een fractie van een seconde, maar binnen deze grens blijven, is minimale feedback aangewezen om aan te geven dat het systeem actief is.
- **10 seconden - maximale aandachtsspanne:**
Dit vormt de bovengrens waarbij de gebruiker betrokken blijft bij de huidige taak. Bij langere wachttijden neemt de kans toe dat de gebruiker de interactie onderbreekt. In dergelijke gevallen is expliciete feedback, zoals een voortgangsindicator, noodzakelijk om onzekerheid te verminderen en de gebruikservaring te ondersteunen.

Op basis van deze analyse kan worden geconcludeerd dat real-time binnen de context van een webgebaseerd monitoringdashboard voor autonome metselrobots voornamelijk als soft real-time moet worden geïnterpreteerd [1], [4]. De nadruk ligt hierbij niet op directe gebruikersinteractie, maar op het tijdig beschikbaar stellen van actuele informatie aan de gebruiker.

Aangezien het dashboard uitsluitend dient voor monitoring en geen actieve besturing van de robot bevat, ligt de focus op het behouden van een accuraat en consistent beeld van de toestand van de robot. In dit kader is het essentieel dat statusupdates, foutmeldingen en voortgangsinformatie met een lage en stabiele latentie worden weergegeven, idealiter binnen een tijdsvenster van enkele honderden milliseconden tot maximaal één seconde [4], [3].

Deze vereisten vormen de basis voor de verdere analyse van geschikte real-time update-strategieën en systeemarchitecturen binnen dit onderzoek.

7 Technologische verkenning en impactanalyse van update-strategieën

De meeste real-time updatestrategieën vallen in drie families: pull (polling), server push over HTTP (SSE/streaming), en bidirectionele sessies (WebSocket/WebRTC/RPC-varianten) [4]. Standaarden en kernspecificaties komen vooral uit IETF (RFC's zoals WebSocket, HTTP/2/3, QUIC, TLS), W3C/WHATWG (SSE/EventSource, WebTransport, Service Workers), en OASIS Open (MQTT) [4].

Om de impact van deze strategieën op netwerkprestaties en efficiëntie te begrijpen, kunnen ze als volgt worden gekarakteriseerd [4]:

- **Pull (Polling):**
Periodieke HTTP-verzoeken van client naar server. Eenvoudig te implementeren, maar leidt tot verhoogde netwerkoverhead (headers, connectiebeheer) en onnodige vertraging door lege aanvragen [4].
- **Server Push (SSE/Streaming):**
Efficiënt voor eenrichtingsverkeer. Zodra een initiële verbinding is opgezet, kan de server proactief data versturen, waardoor onnodige HTTP-verzoeken worden vermeden [4].
- **Bidirectionele sessies (bijv. WebSocket):**
Geschikt voor real-time tweeweginteractie met lage latentie (bijv. chat, IoT). Een persistente verbinding maakt directe communicatie in beide richtingen mogelijk, met minder overhead per bericht dan bij traditionele HTTP-requests [4].

7.1 Polling

Polling is de meest traditionele methode om gegevens op te halen in webapplicaties. Het volgt een strikt request-response-model waarbij de client (bijvoorbeeld een dashboard) het initiatief neemt en de server periodiek bevraagt of er nieuwe data beschikbaar is [4]. Binnen deze strategie worden twee varianten onderscheiden: *short polling* en *long polling*.

7.1.1 Short polling

Bij short polling verzendt de client met vaste, vooraf ingestelde tussenpozen HTTP-verzoeken. De server verwerkt elk verzoek synchroon en stuurt onmiddellijk het antwoord terug, ofwel de meest recente gegevens, ofwel een leeg antwoord dat aangeeft dat er geen nieuwe data zijn [4]. Deze aanpak is eenvoudig te implementeren binnen de bestaande HTTP-infrastructuur en houdt de complexiteit van de applicatie laag [4].

Voor real-time of near-real-time toepassingen kent short polling echter aanzienlijke nadelen. Ten eerste is er een inherente vertraging: nieuwe gegevens kunnen pas bij het volgende geplande interval worden opgehaald, wat een afweging oplevert tussen reactietijd en verzoekfrequentie [4]. Ten tweede veroorzaakt de methode een hoge netwerkbelasting. Elk verzoek brengt de overdracht van HTTP-headers met zich mee en, in veel configuraties, het opzetten van een nieuwe TCP-verbinding, zelfs wanneer er geen nieuwe data zijn [4]. Bij een groot aantal clients leidt dit tot onnodige serverbelasting en verspilde bandbreedte. Grigorik [4] benadrukt dat dergelijk *busy waiting* ongeschikt is voor scenario's waarin een snelle levering met minimaal resourcegebruik vereist is.

7.1.2 Long polling

Long polling is ontwikkeld als optimalisatie om de tekortkomingen van short polling te verhelpen. De client stuurt opnieuw een HTTP-verzoek, maar als er geen data direct beschikbaar zijn, houdt de server de verbinding open. De server wacht tot er een update plaatsvindt of tot een vooraf ingestelde time-out wordt bereikt, en stuurt pas dan een antwoord [4]. Zodra de client een antwoord ontvangt (met data of als gevolg van een time-out), stuurt deze onmiddellijk een nieuw verzoek, waardoor de cyclus in stand blijft [4].

Dit mechanisme simuleert een persistente verbinding en stelt de server in staat om updates met een aanzienlijk lagere latentie naar de client te sturen dan bij short polling, terwijl het aantal lege verzoeken vermindert [4]. De IETF-analyse in RFC 6202 erkent long polling als een pragmatische techniek om de responsiviteit te verbeteren in omgevingen waar echte streaming of bidirectionele protocollen (zoals WebSockets) niet beschikbaar of praktisch zijn [5]. Toch wijst de RFC ook op blijvende tekortkomingen: elke berichtuitwisseling brengt nog steeds de overhead van een volledige HTTP-request-response-cyclus met zich mee, inclusief headerverwerking en, als HTTP keep-alive niet optimaal is ingesteld, het heropbouwen van de TCP-verbinding [5]. Daarnaast kan long polling schaalbaarheidsproblemen veroorzaken aan de serverzijde, omdat elke client een verbinding gedurende langere tijd openhoudt, wat geheugen en bestandsdescriptoren consumeert [4], [5].

7.1.3 Vergelijkende analyse en toepasbaarheid

De volgende tabel vat de belangrijkste verschillen tussen short polling en long polling samen:

Tabel 1 Vergelijking van short polling en long polling

Aspect	Short polling	Long polling
Latentie	Beperkt door polling-interval; doorgaans hoog	Vrijwel onmiddellijk (behalve bij time-out)
Serverbelasting	Hoog door frequente, vaak lege verzoeken	Matig; verbindingen staan open, maar minder verzoeken
Netwerkoverhead	Hoog (headers, TCP-opbouw per interval)	Lager per bericht, maar overhead per levering
Implementatie	Zeer eenvoudig	Iets complexer (time-out-afhandeling)
Geschiktheid voor real-time	Slecht	Matig (pseudo-real-time)

Ondanks deze beperkingen blijft polling relevant in toepassingen met lage updatefrequentie of beperkte schaal, waar de eenvoud en brede compatibiliteit opwegen tegen de inefficiënties.

In de context van het monitoringdashboard van Wally Automation, dat een efficiënte levering vereist van hoogfrequente, tijdskritische robotdata, vertonen beide polling-technieken fundamentele beperkingen. Short polling introduceert ofwel te veel latentie, of leidt bij kortere intervallen tot een onhoudbare belasting van netwerk en server. Long polling vermindert de latentie, maar behoudt de overhead van herhaalde HTTP-handshakes en biedt geen echte server-gedreven streaming. Voor dergelijke real-time scenario's met hoge doorvoer zijn geavanceerdere paradigma's zoals WebSockets of Server-Sent Events (SSE) beter geschikt, omdat zij een persistente, laag-overhead kanaal bieden waarover data direct kunnen worden gepusht zodra ze beschikbaar zijn [4], [5]. Dit wordt bevestigd door empirisch onderzoek naar WebSocket-adoptie, dat aantoont dat deze technologie in de praktijk op grote schaal wordt ingezet voor real-time webtoepassingen [2].

Hoewel polling eenvoudig te implementeren is, vertoont deze methode duidelijke beperkingen in real-time scenario's, wat de noodzaak voor push-gebaseerde technieken benadrukt.

7.2 Server Push (SSE)

Server-Sent Events (SSE) vormen een real-time communicatiemechanisme waarbij de server proactief data naar de client kan sturen via een persistente HTTP-verbinding. In tegenstelling tot bidirectionele protocollen zoals WebSockets is SSE beperkt tot unidirectionele communicatie, waarbij enkel de server berichten initieert. De specificatie van SSE is vastgelegd binnen de HTML Living Standard van de WHATWG [7].

Bij SSE wordt een langdurige HTTP-verbinding opgezet via het EventSource-mechanisme, waarbij de server een continue stroom van tekstgebaseerde events naar de client stuurt. Deze aanpak vermijdt de noodzaak van herhaaldelijke request-responsecycli en reduceert daarmee de overhead die typisch is voor pollingmechanismen.

7.2.1 SSE binnen HTTP/1.1

Binnen een klassieke HTTP/1.1-context biedt SSE een efficiënter alternatief voor polling. Waar polling voor elke update een nieuwe HTTP-request vereist, blijft bij SSE de verbinding open en kunnen updates onmiddellijk worden doorgestuurd zodra ze beschikbaar zijn. Volgens Ilya Grigorik vermindert dit het aantal redundante netwerkverzoeken en verlaagt het de latentie in vergelijking met polling [4].

Desondanks kent SSE onder HTTP/1.1 belangrijke beperkingen. Browsers hanteren doorgaans een limiet op het aantal gelijktijdige verbindingen per domein, wat de schaalbaarheid kan beperken in toepassingen met meerdere parallelle datastromen. Aangezien elke SSE-stream een afzonderlijke HTTP-verbinding vereist, kan dit leiden tot resourcebeperkingen aan zowel client- als serverzijde [4].

7.2.2 SSE binnen HTTP/2

Met de introductie van HTTP/2, gespecificeerd door de Internet Engineering Task Force in RFC 9113 [8], worden een aantal structurele beperkingen van HTTP/1.1 aangepakt. HTTP/2 introduceert onder meer multiplexing, waarbij meerdere logische datastromen gelijktijdig over één enkele TCP-verbinding kunnen worden verzonden.

In deze context kan SSE efficiënter worden toegepast, aangezien meerdere eventstreams niet langer afzonderlijke TCP-verbindingen vereisen, maar als parallelle streams binnen één verbinding kunnen worden afgehandeld. Dit vermindert de overhead van connectiebeheer en verhoogt de schaalbaarheid van SSE-gebaseerde systemen.

Hoewel HTTP/2 de prestaties en efficiëntie van SSE aanzienlijk verbetert, blijven de fundamentele eigenschappen van SSE behouden. De communicatie blijft unidirectioneel en gebaseerd op HTTP-semantiek, wat betekent dat SSE minder geschikt is voor scenario's waarin interactieve of bidirectionele communicatie vereist is.

7.2.3 Positionering ten opzichte van andere strategieën

SSE bevindt zich conceptueel tussen polling en volledig bidirectionele protocollen zoals WebSockets. In vergelijking met polling elimineert SSE de noodzaak voor herhaalde verzoeken en verlaagt het de latentie en netwerkoverhead. In vergelijking met WebSockets blijft SSE echter beperkter, aangezien het geen full-duplex communicatie ondersteunt.

Hierdoor is SSE vooral geschikt voor toepassingen waarbij de server frequent updates moet versturen naar de client, maar waarbij geen terugkerende communicatie vanuit de client vereist is, zoals notificatiesystemen of monitoringdashboards.

7.3 Bidirectioneel

Waar technieken zoals Server-Sent Events (SSE) efficiënt zijn in het asynchroon pushen van data van de server naar de client, blijven zij fundamenteel asymmetrisch en unidirectioneel [7]. Binnen de context van geavanceerde webtoepassingen, zoals het robotmonitoringdashboard van Wally Automation, kan er echter behoefte ontstaan aan naadloze tweerichtingscommunicatie. Bidirectionele protocollen stellen zowel de client als de server in staat om op elk gewenst moment en onafhankelijk van elkaar berichten te initiëren en te verzenden.

Hoewel het huidige dashboard primair dient voor monitoring, vormt een bidirectioneel communicatiekanaal een robuuste basis voor toekomstige uitbreidingen. Denk hierbij aan interactieve functionaliteiten zoals het verzenden van noodstopcommando's, het dynamisch aanpassen van de configuratie of het opvragen van specifieke logs bij de autonome metselrobot.

7.3.1 WebSockets

Het WebSocket-protocol is gestandaardiseerd door de Internet Engineering Task Force (IETF) in RFC 6455 [6]. Het protocol is ontworpen om full-duplex communicatie te faciliteren over een enkele, aanhoudende TCP-verbinding [6]. Dit vormt een aanzienlijke breuk met het traditionele request-response model van HTTP en biedt een krachtig alternatief voor technieken zoals long polling [5]. Recente studies bevestigen bovendien dat WebSockets breed worden toegepast in moderne real-time webapplicaties, wat hun praktische relevantie voor dergelijke toepassingen benadrukt [2].

De HTTP Upgrade Handshake

Een WebSocket-verbinding begint initieel als een regulier HTTP-verzoek. De client stuurt een request met een specifieke Upgrade-header om aan de server te verzoeken het communicatieprotocol te wisselen van HTTP naar WebSocket [6]. Als de server dit ondersteunt en accepteert, wordt de verbinding geüpgraded en blijft de onderliggende TCP-socket openstaan voor verdere communicatie.

Deze aanpak heeft een cruciaal voordeel voor real-time datastromen: zodra de verbinding tot stand is gebracht, kunnen berichten heen en weer worden gestuurd zonder de aanzienlijke overhead van HTTP-headers die bij elke request in traditionele polling-mechanismen wordt meegestuurd [4]. Dit verlaagt de *end-to-end* latentie dramatisch en reduceert de belasting op het netwerk, wat essentieel is voor omgevingen met hoge updatefrequenties [4].

Beperkingen en Overwegingen

Onlangs de uitstekende prestaties op het gebied van latentie en netwerkoverhead, brengt het gebruik van WebSockets voor het dashboard van Wally Automation ook uitdagingen met zich mee [2]:

- **Geen ingebouwde herverbinding:** In tegenstelling tot SSE, dat via de EventSource API in de browser native ondersteuning biedt voor automatische herverbindingen [7], vereist de WebSocket API dat ontwikkelaars zelf logica implementeren om netwerkonderbrekingen op te vangen. Dit is een belangrijk aandachtspunt op een bouwwerf waar netwerkverbindingen instabiel kunnen zijn.
- **Databindheid:** WebSockets fungeren als een zeer dunne transportlaag. Het protocol verstuurt ruwe tekst- of binaire frames, maar biedt zelf geen ingebouwde mechanismen voor berichtrouting of Quality of Service (QoS) garanties [2], [6]. Dit betekent dat eventuele semantiek rondom het afleveren of filteren van specifieke robotdata volledig op applicatieniveau moet worden afgehandeld.
- **Architecturale complexiteit door protocol-heterogeniteit:** Binnen het bestaande robotica-ecosysteem van Wally Automation maken de autonome robots al gebruik van sterk geoptimaliseerde protocollen (gRPC) voor hun interne en missiekritieke backend-communicatie. Het rechtstreeks integreren van WebSockets op de robot ten behoeve van een webdashboard zou betekenen dat er een tweede, afzonderlijke netwerkstack op de robot moet draaien en onderhouden worden. Dit verhoogt niet alleen de complexiteit van de codebase, maar consumeert ook onnodige systeembronnen op de robot. Om dergelijke protocol-heterogeniteit op het eindapparaat te vermijden, ontstaat vaak de noodzaak voor een architecturale tussenlaag (zoals een gateway of Backend-for-Frontend) die de communicatie vertaalt.

7.4 MQTT (Message Queuing Telemetry Transport)

MQTT (Message Queuing Telemetry Transport) is een lichtgewicht messagingprotocol, ontworpen voor efficiënte communicatie in omgevingen met beperkte bandbreedte, hoge latentie of onbetrouwbare netwerkverbindingen. Het protocol werd gestandaardiseerd door OASIS en is specifiek gericht op toepassingen binnen het Internet of Things (IoT) en gedistribueerde systemen [9].

7.4.1 Werking en architectuur

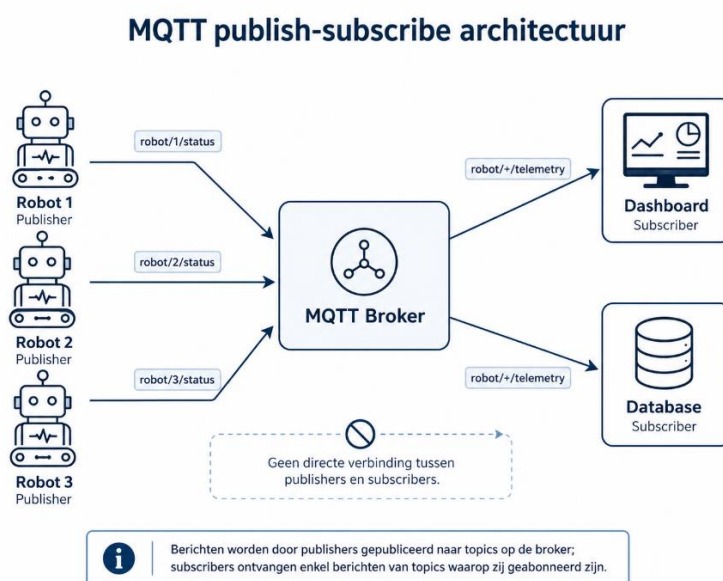
MQTT maakt gebruik van een publish-subscribe model, waarbij communicatie verloopt via een centrale broker. In tegenstelling tot traditionele client-serverarchitecturen communiceren clients niet rechtstreeks met elkaar, maar via deze tussenlaag [9].

Binnen dit model kunnen drie rollen worden onderscheiden:

- **Publisher:** een client die berichten verstuurt naar een bepaald topic
- **Subscriber:** een client die zich abonneert op één of meerdere topics om berichten te ontvangen
- **Broker:** een centrale server die berichten ontvangt, filtert en distribueert naar de juiste subscribers

Wanneer een publisher een bericht verzendt naar een specifiek topic, zorgt de broker ervoor dat alle subscribers die op dit topic geabonneerd zijn het bericht ontvangen. Deze ontkoppeling tussen verzender en ontvanger verhoogt de flexibiliteit en schaalbaarheid van het systeem [9].

Daarnaast ondersteunt MQTT hiërarchische topicstructuren (bijvoorbeeld robot/1/status of project/alpha/errors), waardoor berichten efficiënt kunnen worden gefilterd en georganiseerd [9]. Subscribers kunnen daarbij gebruikmaken van wildcards, zoals het '+'-teken, om zich te abonneren op alle topics op één niveau (bijvoorbeeld robot/+/telemetry voor de telemetry van elke robot) [9]. Figuur 1 toont deze publish-subscribe architectuur toegepast op de context van Wally Automation.



Figuur 1 MQTT publish-subscribe architectuur

In het diagram publiceren meerdere robots naar hun eigen status-topics op de broker, terwijl het dashboard en de database zich als subscribers abonneren op de relevante topics. Publishers en subscribers communiceren nooit rechtstreeks; alle berichten verlopen via de broker, wat de architecturale ontkoppeling van het systeem visueel onderstreept.

7.4.2 Quality of Service (QoS)

Een belangrijk kenmerk van MQTT is de ondersteuning van verschillende Quality of Service (QoS)-niveaus, waarmee de betrouwbaarheid van berichtaflevering kan worden afgestemd op de noden van de toepassing [9]:

- **QoS 0 - At most once:** Berichten worden éénmalig verzonden zonder bevestiging. Dit biedt de laagste latentie en minimale overhead, maar er is geen garantie dat het bericht effectief aankomt.
- **QoS 1 - At least once:** Berichten worden opnieuw verzonden totdat een bevestiging wordt ontvangen. Dit garandeert aflevering, maar kan leiden tot dubbele berichten.
- **QoS 2 - Exactly once:** Berichten worden gegarandeerd exact één keer afgeleverd via een meerfasige handshake. Dit biedt de hoogste betrouwbaarheid, maar introduceert extra latentie en netwerkoverhead.

Deze niveaus maken het mogelijk om per type data een afweging te maken tussen betrouwbaarheid en performantie, wat essentieel is in real-time systemen.

7.4.3 Efficiëntie en performantie

MQTT is ontworpen met efficiëntie als prioriteit. Het protocol gebruikt een minimale headerstructuur (slechts enkele bytes), waardoor de netwerkoverhead aanzienlijk lager ligt dan bij HTTP-gebaseerde communicatie. Seoane et al. [10] bevestigen dat MQTT in veel IoT-scenario's efficiënter omgaat met bandbreedte en CPU-gebruik dan alternatieve protocollen, terwijl het toch een goede balans behoudt tussen latentie en betrouwbaarheid.

Daarnaast kan MQTT gebruikmaken van een persistente TCP-verbinding tussen client en broker, waardoor berichten snel kunnen worden verzonden zonder telkens een nieuwe connectie op te zetten. Dit resulteert in een lagere end-to-end latentie in vergelijking met pollingmechanismen.

7.4.4 Betrouwbaarheid en robuustheid

MQTT bevat verschillende mechanismen om met netwerkproblemen om te gaan:

- **Persistent sessions:** clients kunnen hun sessie behouden, waardoor gemiste berichten alsnog geleverd kunnen worden na herverbinding.
- **Last Will and Testament (LWT):** de broker kan automatisch een bericht versturen wanneer een client onverwacht wegvalt.
- **Retained messages:** de laatste waarde van een topic kan opgeslagen worden, zodat nieuwe subscribers onmiddellijk de meest recente toestand ontvangen.

Deze eigenschappen maken MQTT bijzonder geschikt voor omgevingen met instabiele connectiviteit, zoals bouwwerven.

7.4.5 Positionering binnen de architectuur van Wally Automation

Binnen de context van Wally Automation bevindt MQTT zich op een ander niveau dan technieken zoals WebSockets of SSE. Waar deze laatste vooral gericht zijn op communicatie tussen backend en webclient, fungeert MQTT eerder als een interne messaginglaag tussen systemen.

De autonome robots genereren continu operationele data, die via een publish-subscribe model efficiënt kunnen worden doorgestuurd naar een centrale broker. Vanuit deze broker kunnen verschillende componenten, zoals databases, analysemodules en webservices, zich abonneren op relevante datastromen.

In deze architectuur kan MQTT dienen als een ontkoppelende laag tussen de robot en het dashboard. Een mogelijke opzet is:

- Robots publiceren data naar MQTT-topics
- Backendservices verwerken deze data
- Een gateway vertaalt MQTT-berichten naar WebSockets of SSE voor visualisatie in het dashboard

Deze aanpak voorkomt dat het webdashboard rechtstreeks moet communiceren met de robot of het interne netwerkprotocol, wat de complexiteit en onderhoudbaarheid ten goede komt.

7.4.6 Evaluatie ten opzichte van andere strategieën

In vergelijking met polling, SSE en WebSockets onderscheidt MQTT zich door zijn rol als message broker-protocol:

- In tegenstelling tot polling vermijdt MQTT onnodige verzoeken en reduceert de netwerkoverhead.
- In vergelijking met SSE en WebSockets biedt MQTT ingebouwde ondersteuning voor bericht distributie, buffering en betrouwbaarheid.
- MQTT is echter niet rechtstreeks bruikbaar in de browser zonder extra componenten (zoals een WebSocket-bridge).

Hierdoor is MQTT vooral geschikt als backend-communicatielaag in real-time systemen, terwijl WebSockets of SSE beter aansluiten bij directe communicatie met webclients.

7.5 gRPC (Google Remote Procedure Call)

gRPC is een open-source Remote Procedure Call (RPC)-framework, oorspronkelijk ontwikkeld door Google, dat efficiënte communicatie mogelijk maakt tussen gedistribueerde systemen en microservices [11]. In tegenstelling tot REST, dat gebruikmaakt van HTTP/1.1 met JSON-payloads, is gRPC gebouwd op HTTP/2 en maakt het gebruik van Protocol Buffers (Protobuf) als serialisatieformaat [11], [12]. Binnen de architectuur van Wally Automation vormt gRPC een belangrijk communicatieprotocol binnen de interne systeemarchitectuur van de autonome metselrobots, met name voor de interactie tussen de robot en externe bedieningsinterfaces zoals een tablet, wat het relevant maakt om dit protocol nader te analyseren.

7.5.1 Werking en architectuur

Het kernprincipe van gRPC is het definiëren van services via een Interface Definition Language (IDL), waarbij Protocol Buffers als standaard IDL worden gebruikt [11]. In een .proto-bestand worden zowel de service-interface als de structuur van de berichten gespecificeerd. Op basis van deze definitie genereert een Protocol Buffers-compiler automatisch client- en servercode in de gewenste programmeertaal [11]. Aan serverzijde implementeert de server de gedeclareerde methoden en draait een gRPC-server die binnenkomende aanvragen afhandelt. Aan clientzijde beschikt de client over een zogenaamde stub, een lokaal object dat dezelfde methoden aanbiedt als de service. De client kan deze methoden aanroepen alsof het lokale functieaanroepen betreft, terwijl de stub de parameters verpakt in het juiste Protocol Buffer-berichtformaat en de aanvraag naar de server verzendt [11].

Protocol Buffers gebruiken een compact binair serialisatieformaat, in tegenstelling tot de tekstgebaseerde JSON-representatie die typisch bij REST wordt ingezet. Dit resulteert in aanzienlijk kleinere berichtgroottes en een efficiëntere verwerking (parsing), wat direct bijdraagt aan de hoge prestaties van gRPC [12]. Daarnaast maakt gRPC gebruik van HTTP/2 als onderliggend transportprotocol, waardoor het kan profiteren van functies zoals multiplexing, headercompressie en bidirectionele streaming over één enkele TCP-verbinding [8], [12].

7.5.2 Communicatiepatronen

gRPC ondersteunt vier verschillende communicatiepatronen, wat het framework bijzonder veelzijdig maakt voor uiteenlopende toepassingen [11]:

- **Unary RPC:** Het klassieke request-response model, waarbij de client één aanvraag verstuurt en één antwoord van de server ontvangt. Dit patroon is vergelijkbaar met een traditionele HTTP-aanvraag en is geschikt voor eenvoudige gegevensbevragingen [11].
- **Server Streaming RPC:** De client verstuurt één aanvraag en de server antwoordt met een stroom van meerdere berichten. Dit is bijzonder relevant voor monitoringtoepassingen, waarbij de server continu statusupdates kan doorsturen naar de client [11].
- **Client Streaming RPC:** De client verstuurt een reeks berichten naar de server, die na ontvangst van de volledige stroom één antwoord terugstuurt. Dit patroon is geschikt voor scenario's waarin grote hoeveelheden data in batches worden aangeleverd [11].

- **Bidirectional Streaming RPC:** Zowel de client als de server kunnen onafhankelijk van elkaar een stroom van berichten verzenden. Dit maakt gelijktijdige tweerichtingscommunicatie mogelijk en is het meest geavanceerde patroon dat gRPC aanbiedt [11].

Deze veelzijdigheid in communicatiepatronen onderscheidt gRPC van protocollen zoals REST en SSE, die beperkt zijn tot respectievelijk request-response en unidirectionele server-push. Met name het server streaming patroon is conceptueel vergelijkbaar met SSE, maar met het voordeel van een compacter berichtformaat en de mogelijkheid om bij te schakelen naar bidirectionele communicatie wanneer dat nodig is.

7.5.3 Prestaties en efficiëntie

De prestatiekenmerken van gRPC zijn onderzocht door Hedelin [12] in een vergelijkende benchmarkstudie aan KTH Royal Institute of Technology. In deze studie werden gRPC, REST en SOAP systematisch vergeleken op throughput, latentie en schaalbaarheid binnen een gecontroleerde microservice-omgeving.

De resultaten van deze studie tonen aan dat gRPC een hogere throughput behaalt dan zowel REST als SOAP in de onderzochte context, wat grotendeels toe te schrijven is aan de eerder beschreven eigenschappen van Protocol Buffers en de multiplexingcapaciteiten van HTTP/2 [12]. Daarnaast toonde de studie aan dat gRPC lagere en stabielere latentie biedt dan REST en SOAP, met name bij toenemende belasting [12].

Op het vlak van schaalbaarheid liet gRPC eveneens betere resultaten zien. Dankzij HTTP/2 multiplexing kunnen meerdere gelijktijdige verzoeken over één enkele TCP-verbinding worden afgehandeld, waardoor de overhead van het opzetten van afzonderlijke verbindingen wegvalt [8], [12]. Dit maakt gRPC bijzonder geschikt voor omgevingen met hoge doorvoer en meerdere gelijktijdige datastromen, zoals het geval is bij de operationele monitoring van autonome metselrobots.

7.5.4 Positionering binnen de architectuur van Wally Automation

Binnen het ecosysteem van Wally Automation neemt gRPC een fundamenteel andere positie in dan de eerder besproken update-strategieën. Terwijl WebSockets, SSE en MQTT zich richten op de communicatie tussen backend en webclient of tussen systemen onderling, fungeert gRPC als een belangrijk communicatieprotocol binnen de interne systeemarchitectuur van de autonome metselrobots, met name voor de interactie tussen de robot en externe bedieningsinterfaces zoals een tablet. gRPC is op dit niveau de juiste keuze omdat het lage latentie biedt, het compacte berichtformaat en de ondersteuning voor bidirectionele streaming, eigenschappen die essentieel zijn voor de betrouwbare uitwisseling van operationele robotdata [11], [12].

Het is echter belangrijk op te merken dat gRPC niet rechtstreeks bruikbaar is in de webbrowser zonder aanvullende lagen, aangezien standaard browser-API's geen directe ondersteuning bieden voor het volledige gRPC-protocol [12]. Hoewel er met gRPC-Web een variant bestaat die browsergebaseerde communicatie mogelijk maakt, vereist deze een proxy als tussenlaag. Dit impliceert dat een vertaallaag nodig is om gRPC-data beschikbaar te maken voor webclients, wat in de praktijk vaak resulteert in een hybride architectuur waarbij gRPC de interne robotcommunicatie verzorgt en een protocol zoals WebSockets of SSE de data beschikbaar stelt aan het dashboard.

8 Synthese en aanzet tot het primair onderzoek

De literatuurstudie heeft een overzicht geboden van de voornaamste update-strategieën die beschikbaar zijn voor webgebaseerde monitoringdashboards. Elke strategie is geanalyseerd op haar werking, sterktes en beperkingen, met bijzondere aandacht voor de context van autonome metselrobots.

Uit de analyse blijkt dat polling, hoewel eenvoudig te implementeren, ongeschikt is voor scenario's met hoge updatefrequenties vanwege de inherente latentie en onnodige netwerkbelasting [4], [5]. SSE biedt een efficiënter alternatief voor unidirectionele server-push, met name onder HTTP/2, maar is beperkt tot éénrichtingscommunicatie [7], [8]. WebSockets maken full-duplex communicatie mogelijk met lage latentie [6], maar missen ingebouwde mechanismen voor betrouwbaarheid en berichtrouwing [2], [6]. MQTT voegt daar als publish-subscribe protocol configureerbare betrouwbaarheid en ontkoppeling aan toe, maar vereist een broker en is niet native bruikbaar in de browser [9]. gRPC, ten slotte, biedt hoge prestaties en veelzijdige communicatiepatronen voor de interne systeemcommunicatie, maar is evenmin rechtstreeks inzetbaar in een webomgeving zonder tussenlaag [11], [12].

Een rode draad doorheen de literatuur is dat geen enkele strategie op alle criteria optimaal scoort. Dit versterkt de hypothese dat een hybride aanpak, waarbij event-gedreven data via een push-gebaseerd protocol wordt verzonden en minder tijdkritische informatie via polling wordt opgehaald, de meest geschikte oplossing kan bieden voor het monitoringplatform van Wally Automation.

De theoretische bevindingen bieden echter onvoldoende basis om deze hypothese te bevestigen of te weerleggen. De geraadpleegde bronnen hanteren uiteenlopende testopstellingen, metrics en scenario's, waardoor een directe vergelijking binnen de specifieke context van Wally Automation niet mogelijk is op basis van bestaand onderzoek alleen. Om deze leegte te adresseren wordt in het volgende deel een Proof of Concept (PoC) opgezet, waarin de besproken strategieën onder gecontroleerde en identieke omstandigheden worden vergeleken op end-to-end latentie, netwerkverbruik, betrouwbaarheid en schaalbaarheid.

V. Primair onderzoek

Conclusie

Reflectie

Bronnenlijst

- [1] BlackBerry QNX, “What is Real Time and Why Do I Need It?,” QNX. [Online]. Beschikbaar op: https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.sys_arch/topic/what_is_realtime.html. (Geraadpleegd op: 18 maart 2026).
- [2] P. Murley, Z. Ma, J. Mason, M. Bailey en A. Kharraz, “WebSocket Adoption and the Landscape of the Real-Time Web,” in *Proceedings of the Web Conference 2021 (WWW ’21)*, 2021, pp. 1192–1203. [Online]. Beschikbaar op: https://faculty.cc.gatech.edu/~mbailey/publications/www21_websocket.pdf. (Geraadpleegd op: 3 april 2026).
- [3] J. Nielsen, “Response Times: The 3 Important Limits”, NNGroup. [Online]. Beschikbaar op: <https://www.nngroup.com/articles/response-times-3-important-limits/> (Geraadpleegd op: 19 maart 2026).
- [4] I. Grigorik, *High Performance Browser Networking*, O'Reilly Media, 2013. [Online]. Beschikbaar op: <https://hpbn.co/> (Geraadpleegd op: 20 maart 2026).
- [5] S. Loreto, P. Saint-Andre, S. Salsano, en G. Wilkins, “Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP”, *IETF*, april 2011. [Online]. Beschikbaar op: <https://datatracker.ietf.org/doc/html/rfc6202> (Geraadpleegd op: 31 maart 2026).
- [6] I. Fette, A. Melnikov, “The WebSocket Protocol”, IETF, december 2011. [Online]. Beschikbaar op: <https://datatracker.ietf.org/doc/html/rfc6455> (Geraadpleegd op: 1 april 2026).
- [7] WHATWG, “Server-Sent Events,” *HTML Living Standard*. [Online]. Beschikbaar op: <https://html.spec.whatwg.org/multipage/server-sent-events.html> (Geraadpleegd op: 2 april 2026).
- [8] M. Thomson, C. Benfield, “HTTP/2”, IETF, juni 2022. [Online]. Beschikbaar op: <https://datatracker.ietf.org/doc/html/rfc9113> (Geraadpleegd op: 2 april 2026).
- [9] OASIS, “MQTT Version 3.1.1,” OASIS Standard, 29 oktober 2014. [Online]. Beschikbaar op: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html> (Geraadpleegd op: 3 april 2026).
- [10] V. Seoane, C. Garcia-Rubio, F. Almenares, en C. Campo, “Performance evaluation of CoAP and MQTT with security support for IoT environments,” *Computer Networks*, vol. 197, art. no. 108338, juli 2021. [Online]. Beschikbaar op: <https://www.sciencedirect.com/science/article/pii/S1389128621003364> (Geraadpleegd op: 3 april 2026).
- [11] gRPC.io, “Core concepts, architecture and lifecycle,” gRPC.io. [Online]. Beschikbaar op: <https://grpc.io/docs/what-is-grpc/core-concepts/> (Geraadpleegd op: 3 april 2026).
- [12] M. N. Hedelin, “Benchmarking and performance analysis of communication protocols: A comparative case study of gRPC, REST, and SOAP,” master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, Jun. 2024. [Online]. Beschikbaar op: <https://kth.diva-portal.org/smash/get/diva2:1887929/FULLTEXT01.pdf> (Geraadpleegd op: 3 april 2026).

Bijlagen

- A. Omschrijving Bijlage A**
- B. Omschrijving Bijlage B**
- C. Omschrijving Bijlage C**

A. Omschrijving Bijlage A

B. Omschrijving Bijlage B

C. Omschrijving Bijlage C

